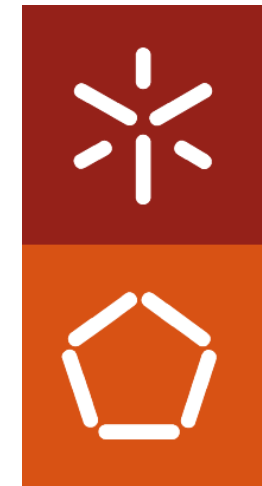


OpenFlow

Gestão e Virtualização de Redes

1º Semestre



OPEN NETWORKING
FOUNDATION

OpenFlow Switch Specification

Version 1.5.1 (Protocol version 0x06)

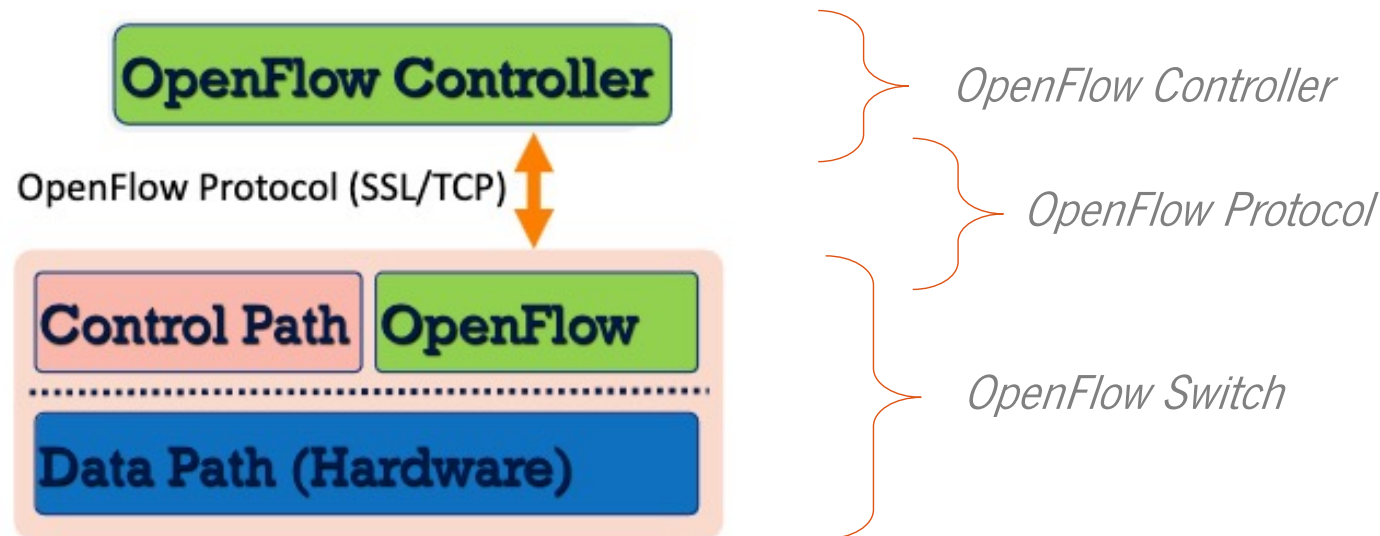
March 26, 2015

<https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.1.pdf>

OpenFlow



- **OpenFlow Switch, OpenFlow Controller e OpenFlow Protocol**



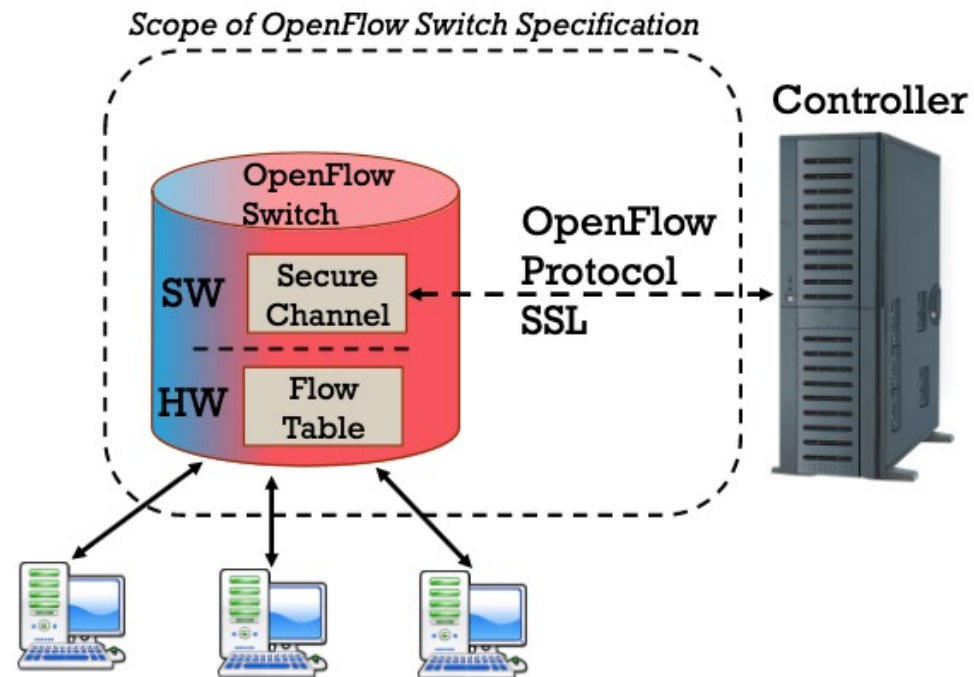
- Requisitos:
 - Uma **lógica comum** em todos os equipamentos (switches, routers, e outros) que os tornem passíveis de ser geridos por um controlador SDN externo
 - Um **protocolo** seguro e normalizado (standard) de comunicação entre controlador e equipamentos

OpenFlow



- **Pipeline OpenFlow**

- Cada *switch* OpenFlow contém múltiplas tabelas de fluxos, e cada tabela múltiplas entradas



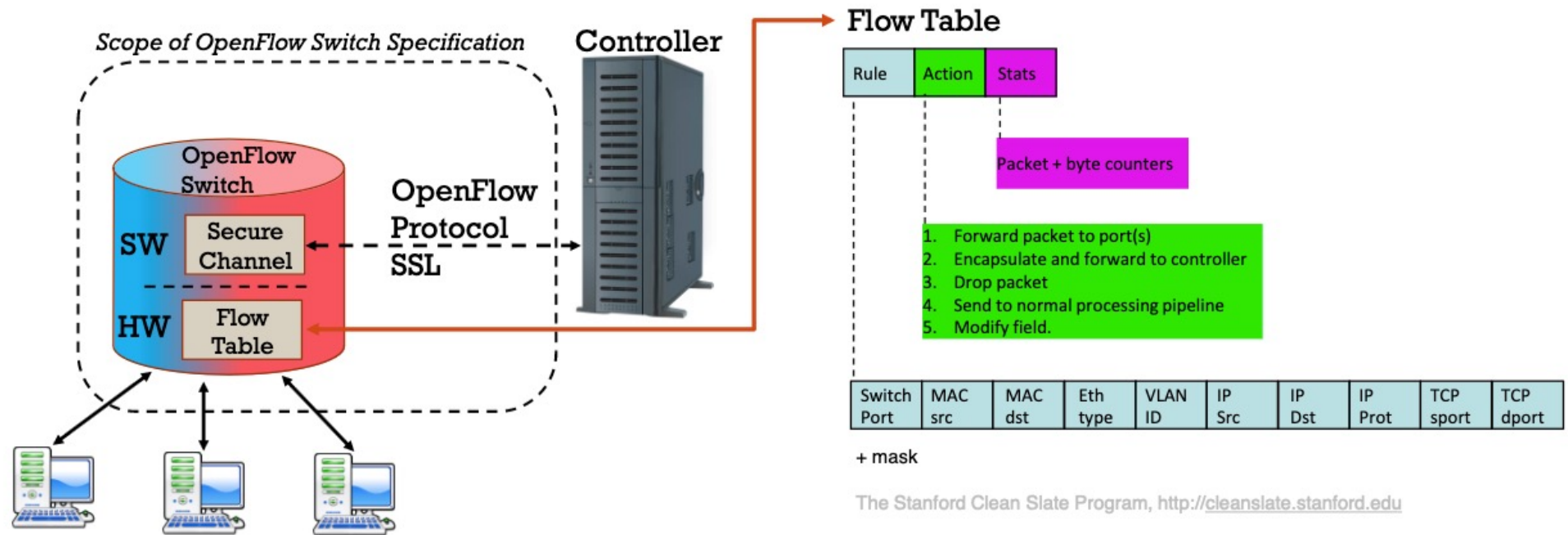
- **Os switches OpenFlow podem ser de dois tipos:**

- **OpenFlow-only:** apenas suportam operação por OpenFlow e todos os pacotes são processados pelo pipeline openflow
- **OpenFlow-hybrid:** suportam OpenFlow, mas permitem modo de operação normal: comutação L2 normal, isolamento VLAN, encaminhamento L3 (IPv4 e IPv6), ACL e QoS

OpenFlow v1.0



- **Versão original** (Clean Slate Program da Universidade Stanford)



- Apenas considerava uma tabela de fluxos por *switch*
- Controlador manipula um conjunto bastante limitado de cabeçalhos: L2 (MAC), L3 (IPv4) e L4 (ports)

OpenFlow v1.0



- **Exemplo (1.0)**

Switching	Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
	*	*	00:1f:..	*	*	*	*	*	*	*	port6
Flow Switching	Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
	port3	00:20..	00:1f..	0800	vlan1	1.2.3.4	5.6.7.8	4	17264	80	port6
Firewall	Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
	*	*	*	*	*	*	*	*	*	22	drop
Routing	Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
	*	*	*	*	*	*	5.6.7.8	*	*	*	port6
VLAN Switching	Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
	*	*	00:1f..	*	vlan1	*	*	*	*	*	port6, port7, port9

Exercício



- **Observe as seguintes regras**

a)

IP src=10.1.1.1 actions=output:1, n_packets=100, n_bytes=5000

IP src= 10.1.1.2 actions=output:2, n_packets=50, n_bytes=2500

IP src=10.1.1.3 actions=output:3, n_packets=25, n_bytes=1250

b)

IP src=10.0.0.1/24, in_port=1 actions=output:1, ...

IP src=10.0.0.1/24, in_port=1 actions=output:4, ...

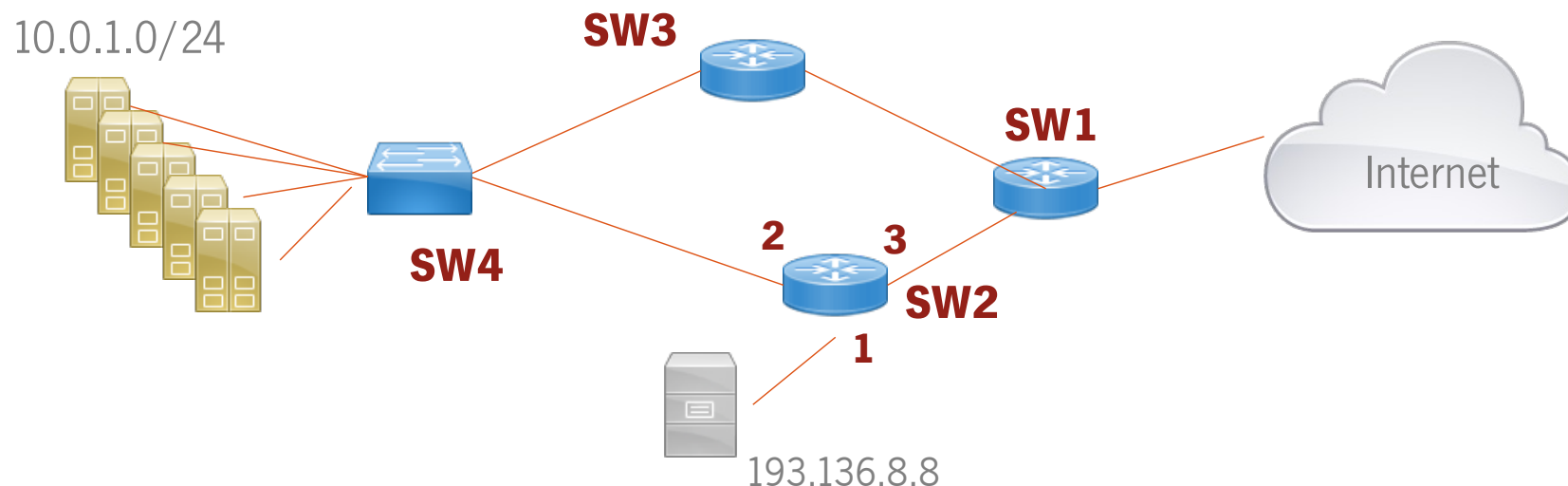
- Que funcionalidade se pretende implementar com este conjunto de regras?
- Detalhe o que acontece aos pacotes que fazem match com estas regras.

Exercício



● Observa a topologia e responda

- O servidor 193.136.8.8 da figura tem um serviço HTTP exposto ao exterior ao qual se pretende garantir conectividade. Todo o restante tráfego do exterior deverá ser bloqueado. Adicionalmente, o mesmo servidor deve ter conectividade interna com a rede da intranet organizacional, sem restrições.

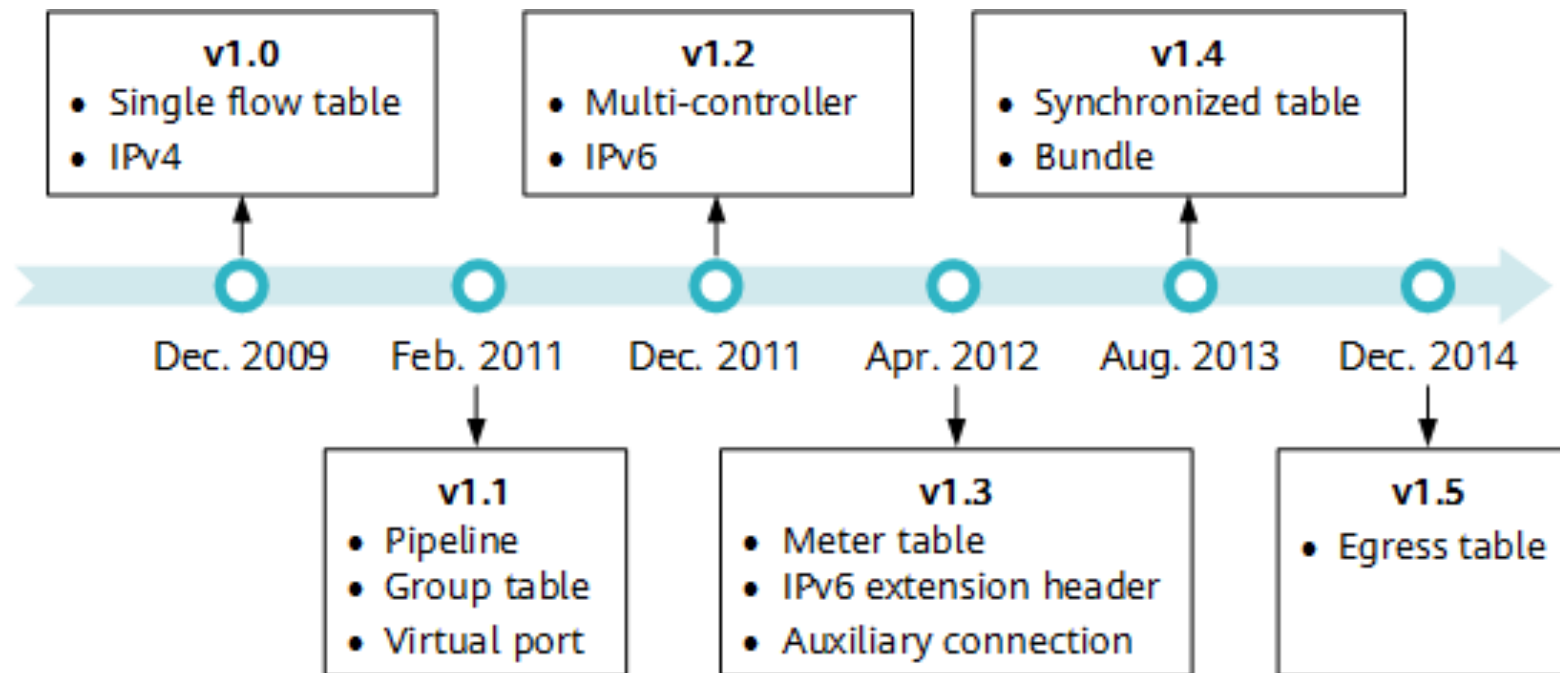


- Os equipamentos da figura podem ser routers, switches L2 ou switches L2/L3?
- Quais as regras OpenFlow que aplicaria no SW2, para concretizar estes requisitos de firewall? Utilize um formato simples para as regras, com apenas dois itens (*match fields* e *action*).
- Esta configuração do SW2 proposta é suficiente ou seria necessário atuar em mais algum equipamento? Justifique.

OpenFlow – evolução



- Desde 2009, novas versões foram surgindo...



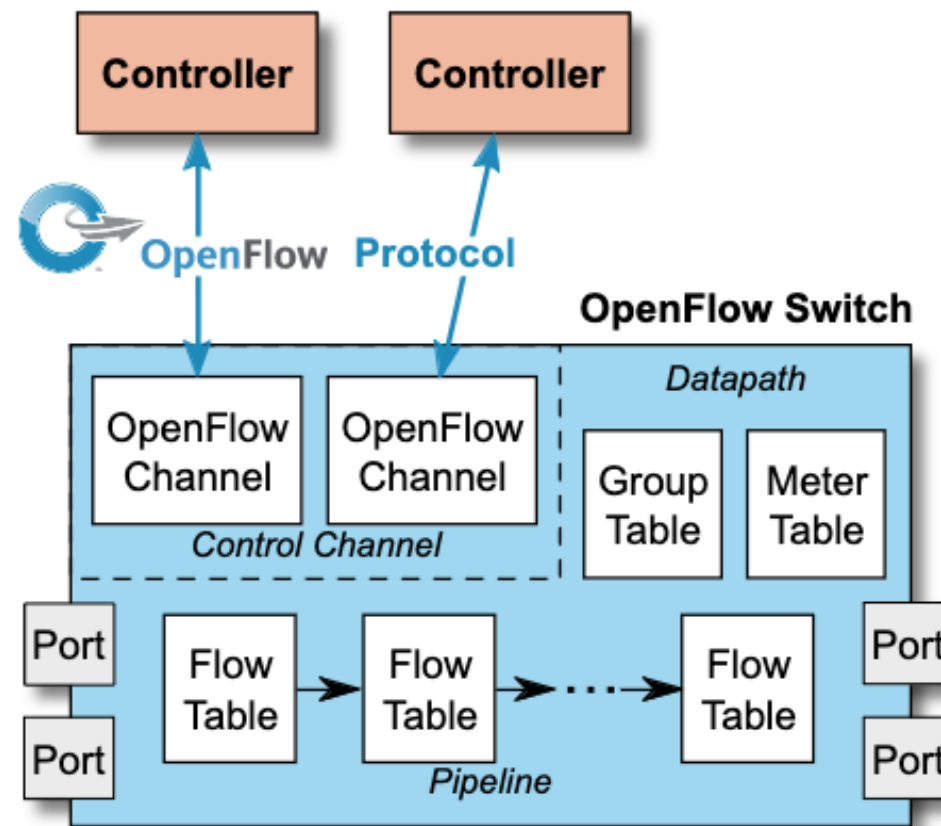
<https://info.support.huawei.com/info-finder/encyclopedia/en/OpenFlow.html>

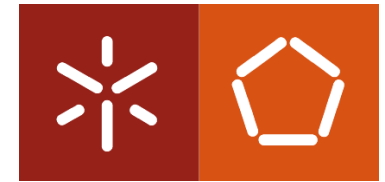
- Novas funcionalidades: Group Tables, Meter Tables, Multi-Controller, Egress Table,
- Versão 1.0 e 1.3 são as mais suportadas e as mais usadas

OpenFlow v1.5.1



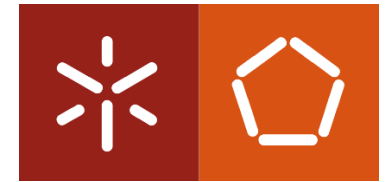
- **OpenFlow Logical Switch: OpenFlow Switch Specification 1.5.1**
 - 26 de março de 2015, Open Networking Foundation





- **Principais componentes e descrição geral**

- Protocolo de comunicação entre o controlador e o switch: *OpenFlow Protocol*
- Cada *switch* comunica com um ou mais controladores, através de um *canal openflow* (*OpenFlow Channel*) usando TLS/TCP/IP
- Uma ou mais *tabelas de fluxos* (*flow tables*) e uma *tabela de grupos* (*group table*) que operam pacote a pacote, pesquisando entradas que façam match com campos do cabeçalho e reenviando os pacotes
- Usando os *canais openflow*, o controlador pode adicionar, alterar e remover *entradas de fluxos* (*flow entries*) nas *tabelas de fluxos*, de duas formas:
 - **Reactivamente**, como resposta a um pacote de dados que entra no switch
 - **Proactivamente**, antes do tráfego ocorrer, e depois do canal openflow se estabelecer
- Cada *tabela de fluxos* contém um conjunto de *entradas de fluxos*
- Cada *entrada de fluxo* consiste em: *campos de verificação* (*match fields*), *contadores* (*counters*) e conjuntos de *instruções* (*instructions*) a aplicar aos pacotes que entram



● Componentes e descrição geral (continuação)

- A verificação dos campos (*match*) começa logo na primeira tabela e prossegue para tabelas adicionais no pipeline
- A verificação tem em conta a ordem por *prioridade*: primeira entrada encontrada em cada uma das tabelas é a usada!
- Se for encontrada uma entrada: as *instruções* associadas a essa entrada são executadas
- Se não for encontrada nenhuma entrada: pode existir uma entrada especial para falhas (*table-miss*) pré configurada que descarte o pacote, o envie ao controlador pelo *canal openflow* estabelecido ou o passe para a próxima *tabela de fluxos* no pipeline
- As instruções associadas a cada *entrada de fluxo* podem:
 - Conter *ações (actions)*: permitem reenviar o pacote para portas, modificação do pacote e processamento na tabela de gupo.
 - Modificar o processamento no pipeline: permitem que os pacotes possam ser passados para outras tabelas no pipeline para processamento adicional, e permitem a inclusão de metadados de informação a ser enviada entre tabelas;



- **Componentes e descrição geral (continuação)**

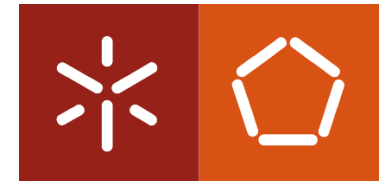
- Uma *entrada de fluxo* pode reenviar o pacote para uma *porta física*, *porta lógica* (túneis, loopback, agregados de links, etc), ou *porta reservada* (controlador, *flooding*, processamento normal, etc)
- *Ações* associadas a entradas também podem remeter para *tabela de grupos*:
 - Grupos representam conjuntos de ações para *flooding*, ou semânticas mais complexas como *mutipath*, *fast reroute* e agregação de links
 - Geralmente é uma abstração que permite gerir um conjunto ações comuns para múltiplos fluxos (ex: todos os fluxos para o mesmo próximo salto)
 - A *tabela de grupos* contém *entradas de grupo*
 - Cada *entrada de grupo* contém uma lista de *ações por atacado* (*action buckets*)
- Há liberdade para implementar internamente o switch OpenFlow da forma mais conveniente, desde que se respeite a semântica de atuação

OpenFlow – Portas



- **Portas OpenFlow Reservadas**

Porta	Descrição
ALL	Representa todas as portas do switch e só pode ser usada como porta de saída, não incluindo a porta de entrada
CONTROLLER	Representa o canal openflow para o controlador e tanto pode ser usada como porta de entrada como de saída
TABLE	Representa o início do pipeline (primeira tabela) e submete o pacote ao processamento para reenvio
IN_PORT	Representa a porta de entrada e só pode ser usada como porta de saída para devolver o pacote
ANY	Porta especial que representa qualquer porta, para match. Não pode ser usada nem como entrada nem como saída. Alguns pedidos do controlador podem especificar a porta e usar ANY como sendo qualquer
UNSET	Quando a porta não foi definida no conjunto de ações e não pode ser usada nem como entrada nem como saída. Usada para referir que o Action-Set não especifica a porta de saída!
LOCAL	Representa a pilha protocolar local do switch (para consumo local)
NORMAL	Representa o comportamento normal do switch, e que não é determinado pelo OpenFlow. Para usar como porta de saída
FLOOD	Representa o flooding tradicional do switch (sem interferência do pipeline openflow) a usar só como saída



- **Alterações**

- Por configuração pode-se adicionar ou remover portas do *switch* a qualquer momento
- As portas estão dependentes do estado da ligação física e podem desligar-se ou ligar-se
- As alterações às portas e ao seu estado não produzem efeitos nas tabelas de fluxos
- Pelo que, qualquer alteração do estado das portas deve originar um evento para que o controlador saiba o que fazer

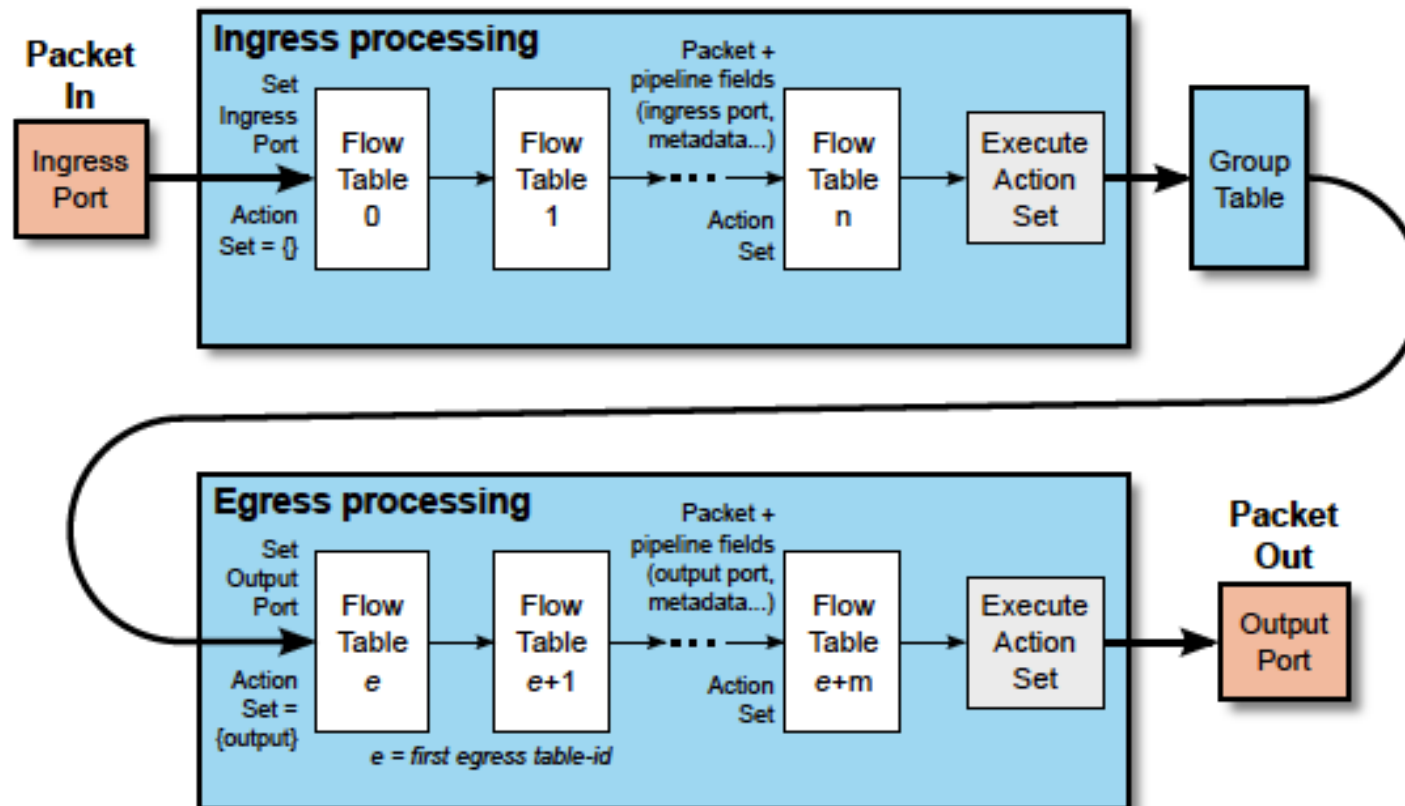
- **Recirculação**

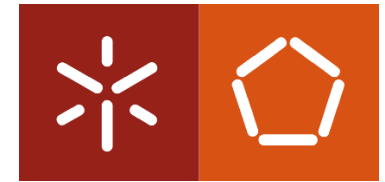
- Normalmente o envio de um pacote por uma porta leva-o para outro equipamento
- No caso das portas lógicas (ex: *loopback*), podem retornar ao mesmo dispositivo, recirculando internamente (eventualmente para encapsulamento ou processamento adicional)
 - Ao reentrar no *switch*, volta ao início do pipeline, mas agora com uma nova IN_PORT
- Pode ser necessário controlar o número de reentradas pela mesma porta para evitar ciclos infinitos

OpenFlow – Tabelas



- Pipeline de processamento

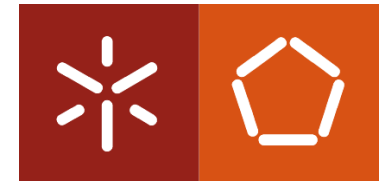




- **Pipeline de processamento:**

- O processamento começa sempre na tabela 0 (zero) – início do pipeline
 - O uso de uma nova tabela depende do *match* e da *acção* definida aí
 - Qualquer regra de uma tabela de fluxos pode remeter o pacote para outra tabela, mas sempre de numeração superior e nunca para trás!
- Ocorre em duas fases: no **ingresso** e no **egresso**!
- Se o resultado do processamento no *ingresso*, for enviar o pacote para uma *porta de saída*, passa-se ao pipeline de *egresso*.
 - Podem não existirem tabelas de fluxos no pipeline de egresso, e nesse caso o pacote é simplesmente enviado pela porta
- Quando o pacote não faz match com nenhuma entrada, ocorre um **table-miss**, e o que se segue depende da configuração da tabela para estes casos
 - As instruções existentes no *table-miss* podem remeter o pacote para outra tabela ou para o controlador, ou simplesmente descartar o pacote (**drop**)
- Sempre que não haja ação definida, o pacote é descartado

OpenFlow – Entradas nas Tabelas



- **Entradas nas tabelas de fluxos**

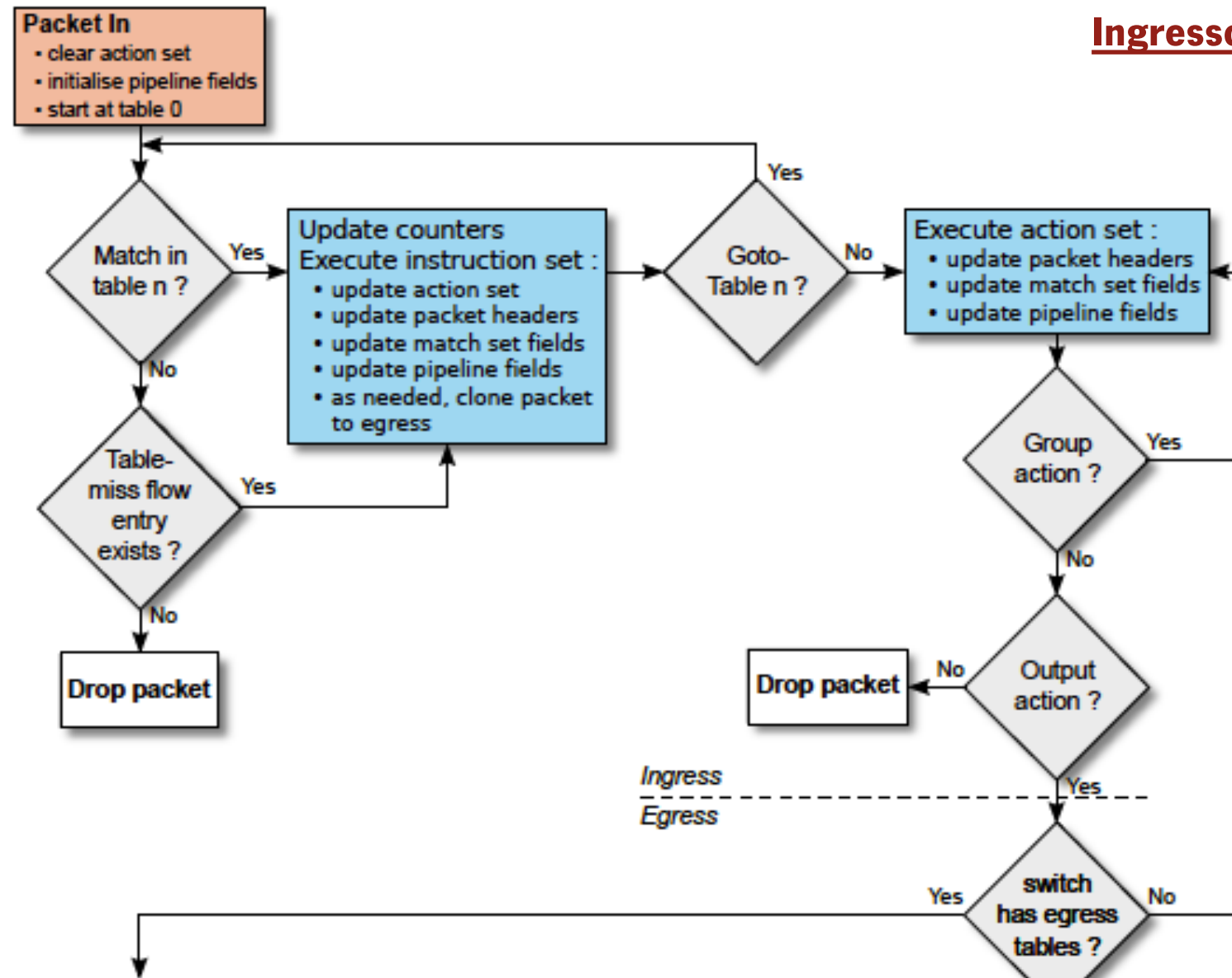
Match Fields	Priority	Counters	Instructions	Timeouts	Cookie	Flags
--------------	----------	----------	--------------	----------	--------	-------

- ***Match fields***: campos a serem verificados. Podem ser campos dos cabeçalhos, campos introduzidos pelo pipeline (metadados) ou a porta de entrada
- ***Priority***: prioridade da entrada
- ***Counters***: contadores a atualizar quando o pacote faz match
- ***Instructions***: modificações no conjunto de acções (*action set*) ou processamento
- ***Timeouts***: tempo máximo de vida até entrada expirar e deixar de ser usada
- ***Cookie***: valor opaco, definido pelo controlador para filtragem e que não afeta o processamento do pacote
- ***Flags***: flags que alteram a forma como a entrada é gerida (ex: OFPFF_SEND_FLOW_REM disparar evento quando a entrada é removida)

OpenFlow – Matching



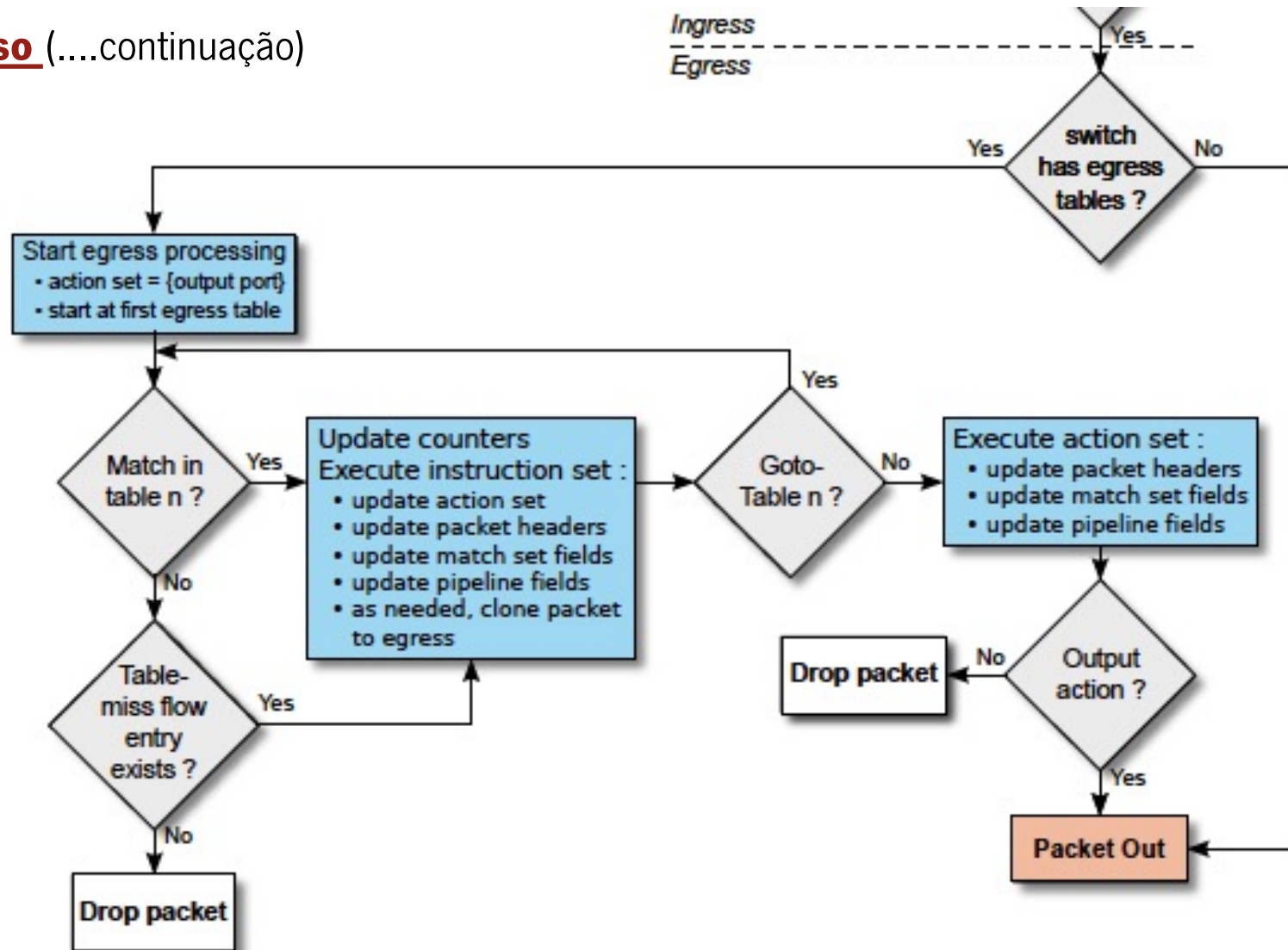
Ingresso (continua...)

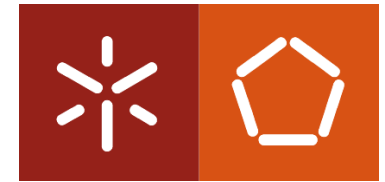


OpenFlow – Matching



Egress (....continuação)





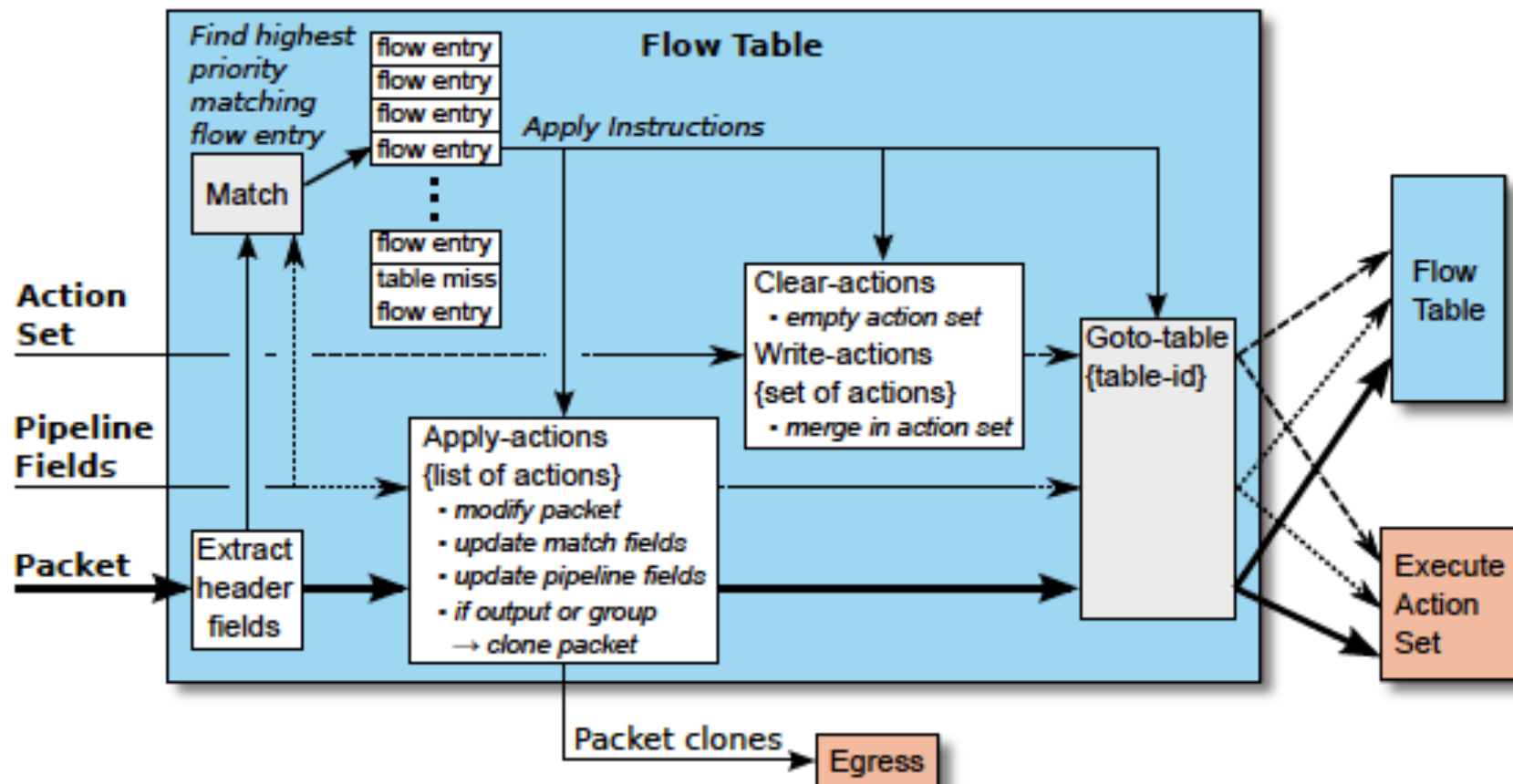
- **Processo de matching:**

- Os campos do pacote são comparados com os das regras
- Os campos dependem do tipo de pacote (IPv4, IPv6, other ...)
- Podem ainda ser usados a porta de entrada e os campos de *metadados*
- Todos os campos devem fazer match
- Quando um campo é omitido todos os valores verificam (ANY)
- Apenas uma entrada faz match – a de mais alta prioridade
- Se fizer match com múltiplas da mesma prioridade, a entrada selecionada é explicitamente indefinida
- Os contadores associados à entrada selecionada devem ser atualizados automaticamente
- As instruções definidas para a entrada selecionada devem ser executadas
- Todas as tabelas devem suportar uma entrada “*table-miss*” (com todos os campos a ANY e prioridade 0 (zero), a mais baixa possível)
- Não existindo entrada table-miss os pacotes são descartados

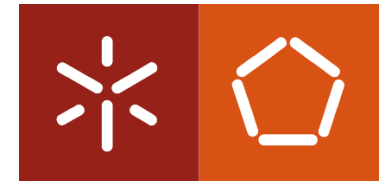
OpenFlow – Instruções



- Matching e execução de instruções numa tabela de fluxos



OpenFlow – Instruções



- **O que se recebe à entrada no processamento de uma tabela de fluxos?**
 - **Pacotes**, aos quais se vão extrair os campos do cabeçalho necessários para fazer a verificação (match para as regras)
 - **Action set**: conjunto de ações que pode se o conjunto vazio (se tabela 0, por exemplo) ou conjunto de ações definidas no processamento em tabelas anteriores
 - **Pipeline fields**: eventuais campos de metadados definidos em fases de processamento anteriores
- **Qual o processamento efetuado?**
 - Extrair campos e ver qual a regra de mais alta prioridade que faz match
 - Executar ações definidas nessa regra que podem ser:
 1. Limpar ações do action-set
 2. Adicionar ações ao action-set
 3. Aplicar imediatamente algumas ações que modifiquem o pacote, atualizem campos do cabeçalho, atualizem campos de metadados, clone pacote para saída



OpenFlow – Instruções

- Cada entrada na tabela inclui um conjunto de instruções a serem executadas

Instrução	Descrição	Observações
<i>Apply-Actions</i>	Aplicar as instruções imediatamente, sem alterar o Action Set.	Optional
<i>Clear-Actions</i>	Limpar todas as ações do Action-Set	Required
<i>Write-Actions</i>	Juntar lista de ações ao Action-Set já existente	Required
<i>Write-Metadata</i>	Escrever metadados no campo próprio (usando a máscara)	Optional
<i>Stat-Trigger</i>	Enviar evento ao controlador se estatísticas passam patamar	Optional
<i>Goto-Table</i>	Indicar próxima tabela no pipeline (número superior)	Required

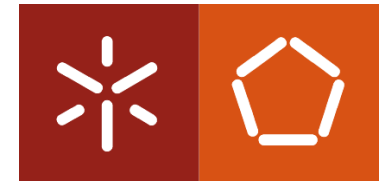
- O conjunto de instruções associado a uma entrada deve ter no máximo uma instrução de cada tipo.
- As instruções são sempre executadas pela ordem definida na tabela acima
- Um *switch* deve rejeitar o fluxo se não conseguir executar todas as instruções!

OpenFlow – *Action-Set*



- **Action-set:** As ações são executadas pela ordem definida na tabela seguinte

Ord	Acção	Descrição
1	copy TTL inwards	Copiar o campo TTL do cabeçalho mais exterior para o seguinte mais interno (ex: túnel IP-sobre-IP, IP-MPLS...)
2	pop	Aplicar todos as ações de pop de etiquetas ao pacote
3	push-MPLS	Aplicar push de etiqueta MPLS
4	push-PBB	Aplicar push de etiqueta PBB (Provider Backbone Bridges)
5	push-VLAN	Aplicar push de etiqueta VLAN
6	copy TTL outwards	Copiar o campo TTL do cabeçalho antes do mais externo para o mais externo
7	decrement-TTL	Decrementar o TTL do pacote
8	set	Aplicar set-field ao pacote
9	qos	Aplicar todas ações QoS, como medidas e set_queue
10	group	Aplicar as ações incluídas no grupo pela ordem desta tabela
11	output	Enviar pacote pela porta referida



- **Action-set**

- Cada pacote tem associado um *Action-set*, inicialmente vazio, que vai passando de tabela em tabela
- O *Action-set* pode ser esvaziado ou acrescentado com instruções definidas nas regras
- A última ação a ser executada é o output
- O output de uma tabela de ingresso é tratado de forma diferente, pois o pacote pode seguir para as tabelas de egresso
- O output de uma tabela de egresso, determina a saída imediata do pacote por essa porta
- A ação output pode referenciar como porta **ALL**, e nesse caso o pacote deve ser clonado para todas as portas de saída
- As ações de um *Action-set* são executadas pela ordem definida na tabela seguinte



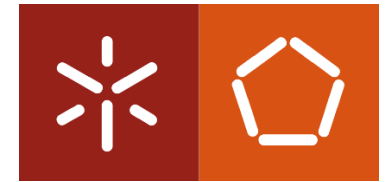
OpenFlow – Ações

- **Listas de Ações**

- A instrução *Apply-Actions* inclui uma lista de ações que são executadas em sequência, pela ordem da lista. O *Action-Set* não é alterado

- **Ações**

Ação	Descrição	Tipo
Output <port>	Enviar pacote para a porta referida	Required
Group <group_id>	Passar pacote para tabela de grupo referida	Required
Drop	Descartar pacote	Required
Set-Queue <queue_id>	Coloca na queue referida da porta de saída	Opcional
Meter <meter_id>	Passar pacote para a tabela de medição	Opcional
Push-Tag/Pop-Tag <tag>	Põe ou tira etiquetas do pacote	Opcional
Set-Field <field> <value>	Modifica campo do cabeçalho	Opcional
Copy-Field <src> <dst>	Copia um campo para outro	Opcional
Change-TTL <ttl>	Altera o TTL	Opcional

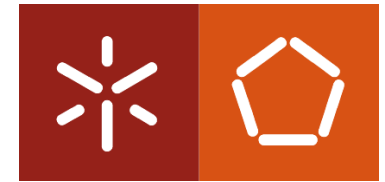


- **Observe as seguinte entrada de uma tabela OpenFlow:**

cookie=0x1, table=0, priority=10, ip, nw_dst=10.0.0.1

actions=set_field:00:11:22:33:44:55->eth_dst,set_field:10.0.0.2->ip_dst,output:1

- a) Identifique os vários campos da entrada na tabela de fluxos.
- b) Quais são os campos do pacote verificados nesta regra?
- c) Qual a condição que deve ser satisfeita para que as ações sejam executadas?
- d) Que modificações são feitas ao pacote?
- e) Não há mais que um tipo de ação na mesma regra, o que supostamente não é possível?
- f) O pacote é passado a outras tabelas, descartado ou reenviado?



- **Observe as seguintes entradas de uma tabela OpenFlow:**

cookie=0x2, table=0, priority=20, ip actions=write_actions(output:3), goto_table:1
cookie=0x3, table=1, priority=30, tcp, tp_dst=80 actions=apply_actions(set_field: 8080->tcp_dst, output:3)

- a) Identifique os vários campos da entrada na tabela de fluxos.
- b) Quais são os campos do pacote verificados nestas regras?
- c) Qual a condição que deve ser satisfeita para que as ações sejam executadas?
- d) Que modificações são feitas ao pacote?
- e) Não há mais que um tipo de ação na mesma regra, o que supostamente não é possível?
- f) O pacote é passado a outras tabelas, descartado ou reenviado?

OpenFlow – contadores



Counter	Bits	
Per Flow Table		
Reference Count (active entries)	32	<i>Required</i>
Packet Lookups	64	<i>Optional</i>
Packet Matches	64	<i>Optional</i>
Per Flow Entry		
Received Packets	64	<i>Optional</i>
Received Bytes	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>
Per Port		
Received Packets	64	<i>Required</i>
Transmitted Packets	64	<i>Required</i>
Received Bytes	64	<i>Optional</i>
Transmitted Bytes	64	<i>Optional</i>
Receive Drops	64	<i>Optional</i>
Transmit Drops	64	<i>Optional</i>
Receive Errors	64	<i>Optional</i>
Transmit Errors	64	<i>Optional</i>
Receive Frame Alignment Errors	64	<i>Optional</i>
Receive Overrun Errors	64	<i>Optional</i>
Receive CRC Errors	64	<i>Optional</i>
Collisions	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>
Per Queue		
Transmit Packets	64	<i>Required</i>
Transmit Bytes	64	<i>Optional</i>
Transmit Overrun Errors	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>

Per Group		
Reference Count (flow entries)	32	<i>Optional</i>
Packet Count	64	<i>Optional</i>
Byte Count	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>
Per Group Bucket		
Packet Count	64	<i>Optional</i>
Byte Count	64	<i>Optional</i>
Per Meter		
Flow Count	32	<i>Optional</i>
Input Packet Count	64	<i>Optional</i>
Input Byte Count	64	<i>Optional</i>
Duration (seconds)	32	<i>Required</i>
Duration (nanoseconds)	32	<i>Optional</i>
Per Meter Band		
In Band Packet Count	64	<i>Optional</i>
In Band Byte Count	64	<i>Optional</i>

OpenFlow – Tabela de grupo



- **Group table:**

- Permite definir novos tipos de reenvio (ex: *select*, *all*, *fast failover*)

Group Identifier	Group Type	Counters	Action Buckets
------------------	------------	----------	----------------

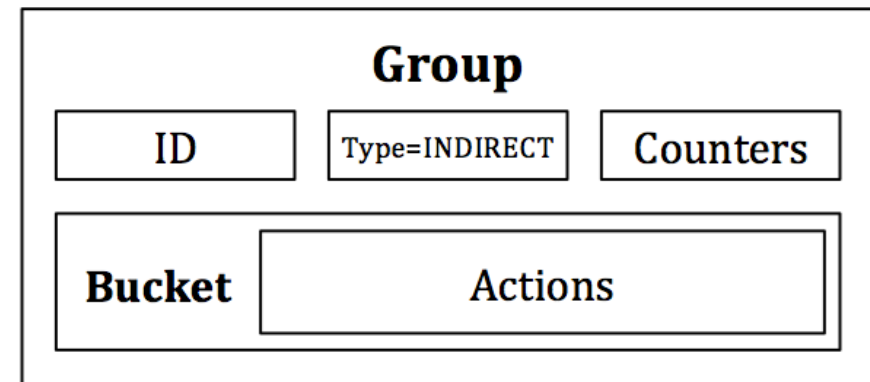
- **Group identifier:** um identificador único de 32 bits
- **Group Type:** define a semântica do grupo (*indirect*, *select*, *all*, *fast failover*)
- **Counters:** contadores atualizados a cada pacote processado
- **Action buckets:** uma lista ordenada de *buckets* com ações, que são executadas com sendo um *Action-set*

OpenFlow – Tabela de grupo



- **Group Type: INDIRECT**

- Para permitir partilha de ações por múltiplos fluxos
- Define apenas um *bucket* pelo que parece estranho que seja de “grupo”, mas é!



- **Exemplos:**

group_id=4,type=indirect,bucket=actions=output:8

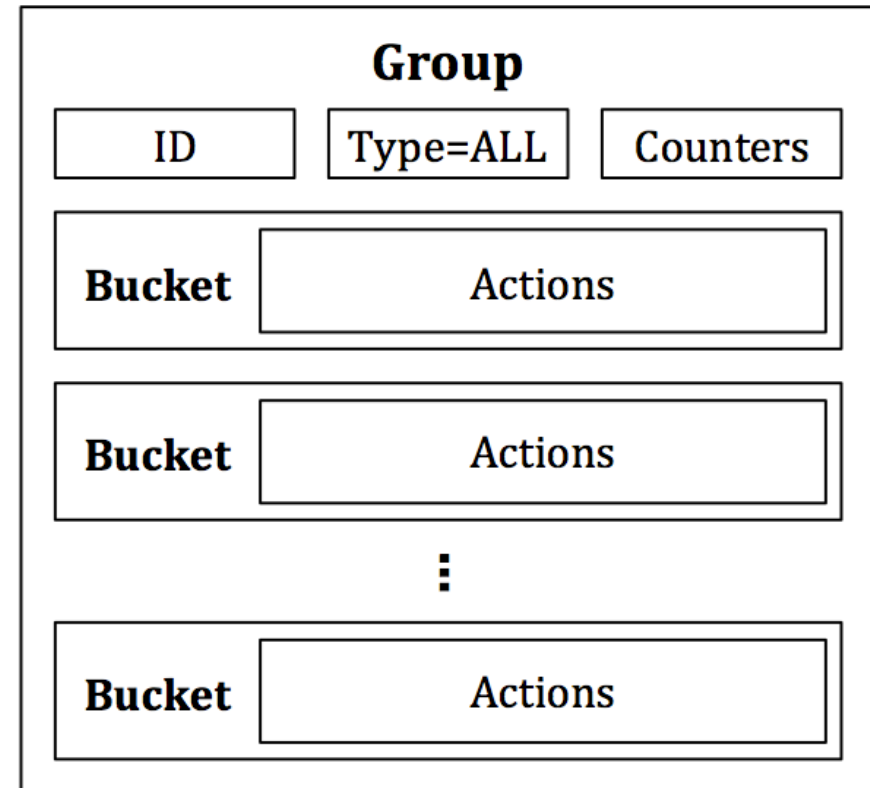
OpenFlow – Tabela de grupo



- **Group Type: ALL**

- Para permitir *Broadcast* ou *Multicast*!
- Todos os *buckets* de ações são executados
- O pacote é clonado para cada *bucket*

- **Exemplos:**



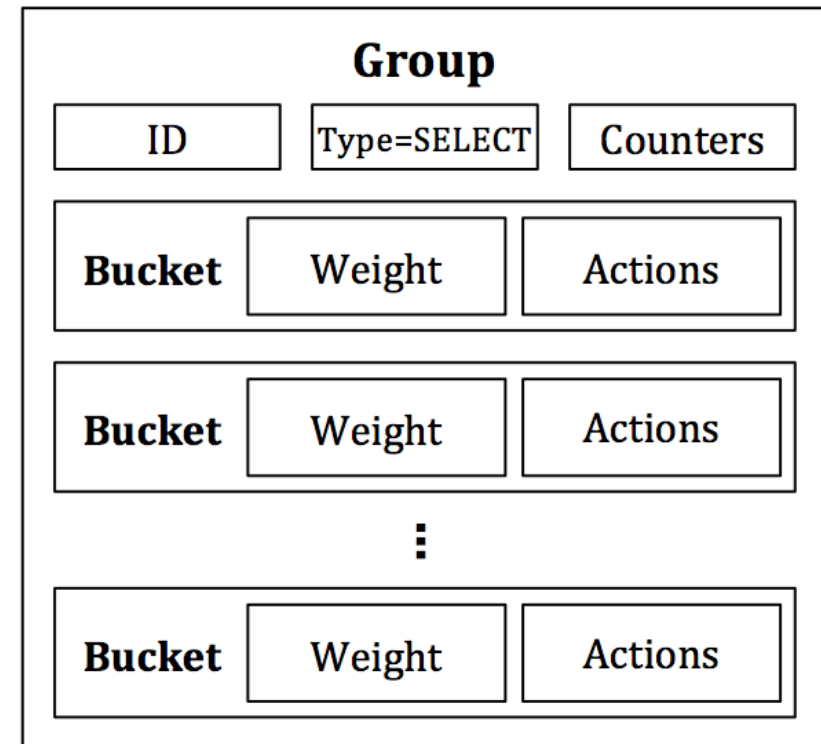
group_id=1,type=all,bucket=actions=output:2,bucket=actions=output:3

OpenFlow – Tabela de grupo



- **Group Type: SELECT**

- Para permitir balanceamento de carga, por exemplo!
- Só um *bucket* é que é executado em cada pacote
- O algoritmo usado pode ser por exemplo um simples *Round-Robin*
- O campo peso determina peso relativo no balanceamento



- **Exemplos:**

group_id=2,type=select,

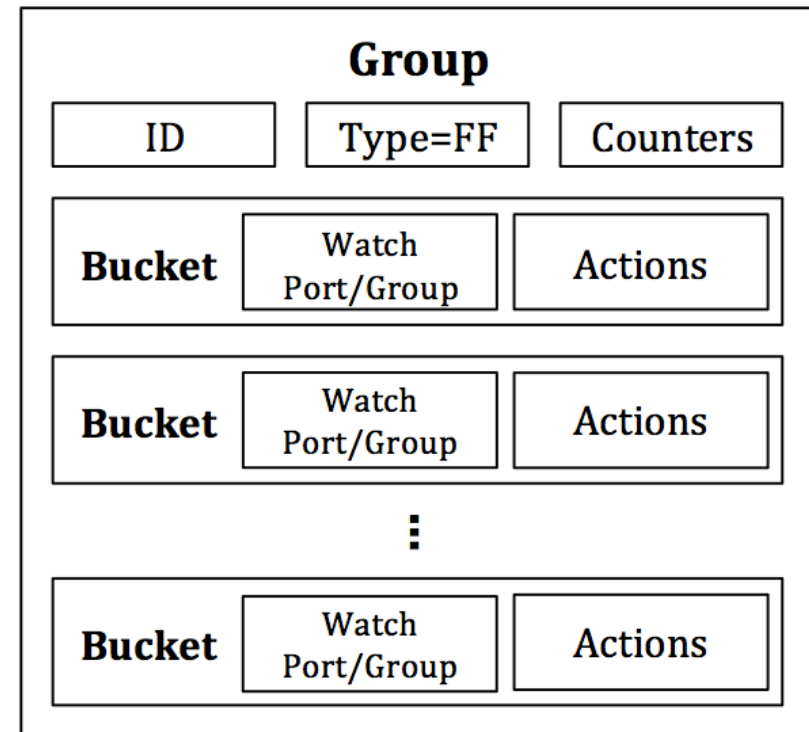
bucket=weight=1,actions=output:4,bucket=weight=2,actions=output:5

OpenFlow – Tabela de grupo



- **Group Type: FAST FAILOVER**

- Para permitir mudança rápida para rota alternativa (*backup*)
- Associado a uma porta
- Monitoriza se a porta está ativa
- Atua sem ir ao controlador em caso de falha de porta
- Só um *bucket* é executado, o primeiro que tiver uma porta "viva" 😊



- **Exemplos:**

group_id=3,type=fast_failover,

bucket=watch_port=6,actions=output:6,bucket=watch_port=7,actions=output:7

OpenFlow – Tabela de medição



- ***Meter table***

- Uma tabela de medição permite controlar o ritmo (débito) por fluxos com o objetivo de restringir a largura de banda consumida (QoS)
- Uma tabela de medição está associada a um fluxo
- Cada entrada da tabela de medição tem os seguintes campos:

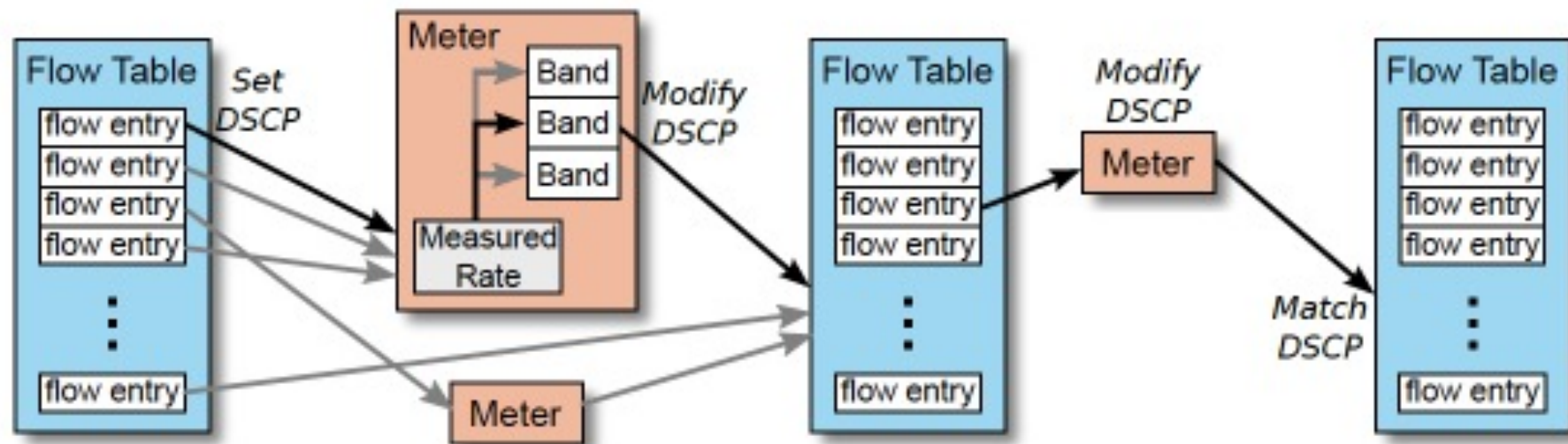
Meter Identifier	Meter Bands	Counters
------------------	-------------	----------

- **Meter identifier:** identificador único de 32 bits
- **Meter bands:** uma lista não ordenada de limites
- **Counters:** contadores associados a esta entrada, atualizados a cada pacote

OpenFlow – Tabela de medição



- Podem-se definir hierarquias de medidores:



- Esta hierarquização permite que numa primeira fase se meçam fluxos individualmente e em fase posteriores de forma agregada

OpenFlow – Tabela de medição



- **Meter-band:**

Band Type	Rate	Burst	Counters	Type specific arguments
-----------	------	-------	----------	-------------------------

- **Band type:** define como os pacotes são processados (*drop*, *dscp remark*, ...)
- **Rate:** taxa definida
- **Burst:** define a granularidade da medida
- **Counters:** contadores que são atualizados a cada pacote que seja medido
- **Type specific arguments:** alguns tipos podem ter argumentos

- **Exemplos de entradas:**

meter=1 drop:rate=1000

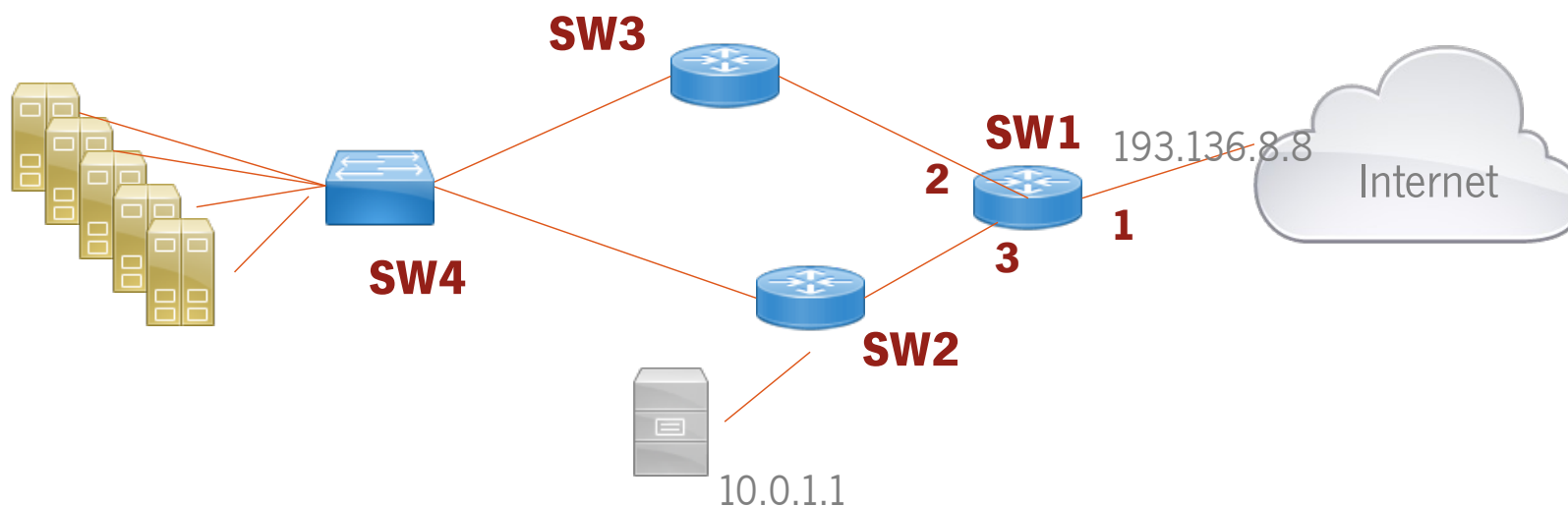
meter=2 drop:rate=5000 dscp_remark:rate=1000 prec_level=1

Exercício



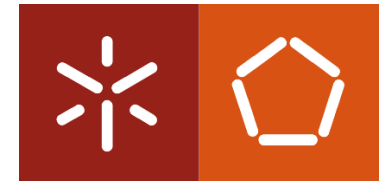
● Pipeline com múltiplas tabelas

- O servidor da figura tem um serviço HTTP exposto ao exterior ao qual se pretende garantir conectividade. Todo o restante tráfego do exterior deverá ser bloqueado. Além disso, é suposto fazer rescrita de endereços no SW1, para que o servidor seja visto do exterior como 193.136.8.8:80 e não como 10.0.1.1:8080 como é acedido internamente.



- Apresente uma solução com três tabelas de fluxos. A primeira (tabela 0) descarta UDP e TCP que não sejam para a porta 80 do servidor 193.136.8.8. A segunda tabela faz a reescrita do IP e da porta (tabela 1). E a terceira e última tabela do pipeline (tabela 2) define a porta de saída.

OpenFlow – Protocolo



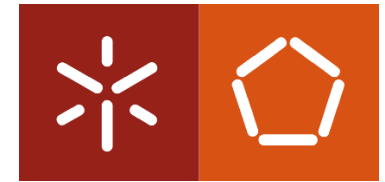
- **Canal de controlo OpenFlow**

- Interface normalizada (standard) entre o controlador e o *switch*
- O *switch* pode suportar mais que um canal de controlo em simultâneo, para múltiplos controladores
- A comunicação com o plano de dados é dependente da implementação e interna ao switch

- **Suporta três tipos de mensagens e múltiplos subtipos:**

- **Controller-to-switch**: iniciadas pelo controlador para monitorizar ou atuar sobre o *switch*
- **Asynchronous**: iniciadas pelo *switch* e usadas para notificar o controlador de eventos internos de mudança de estado
- **Symmetric**: iniciadas quer pelo *switch* quer pelo controlador sem qualquer solicitação prévia

OpenFlow – Protocolo



- **Mensagens Controller-to-switch:**
 - **Features:** o controlador pede a identidade e as funcionalidades OpenFlow que o switch suporta e o switch responde
 - **Configuration:** O controlador consulta ou altera a configuração do switch e o switch responde aos pedidos
 - **Modify-State:** O controlador altera as tabelas de fluxos (adiciona e remove entradas e ações)
 - **Read-State:** Usadas para obter o estado, desde a configuração a várias estatísticas
 - **Packet-out:** Usadas pelo controlador para enviar pacotes para uma porta de saída, depois de os ter recebido por “Packet-in”
 - **Barrier:** pedidos de verificação ou notificações de que as operações foram concluídas
 - Role-Request, Asynchronous-Configuration

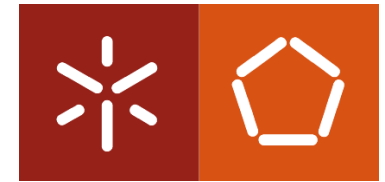
OpenFlow – Protocolo



- **Mensagens Asynchronous:**

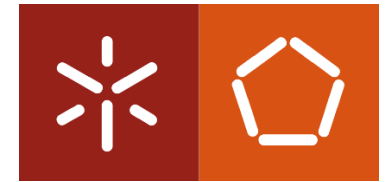
- **Packet-in:** switch envia um pacote ao controlador para ele decidir o que fazer com ele (passagem de controlo)
- **Flow-Removed:** informa o controlador que uma entrada na tabela de fluxos foi removida, quer por ação do controlador, quer por ter expirado, por exemplo
- **Port-Status:** informa o controlador de alteração de estado de uma porta (link down ou up, por exemplo)
- **Role-Status:** Informa o controlador que o seu papel mudou (pode por exemplo ter deixado de ser o controlador master)
- **Controller-Status:** informa o controlador sobre alteração de estados os canais (conexões)
- **Flow-Monitor:** Informa o controlador sobre alterações numa tabela de fluxos

OpenFlow – Protocolo



- **Mensagens Symmetric:**
 - **Hello:** mensagens para iniciar uma conexão
 - **Echo:** as mensagens *echo request* são enviadas por um dos lados e devem ser respondidas com *echo reply*, mantendo conexão viva
 - **Error:** enviadas quer pelo *switch* quer pelo controlador para notificar problemas
 - **Experimenter:** uma forma de adicionar novas funcionalidades experimentais

OpenFlow – Protocolo



- **Canal de controlo OpenFlow**

- Interface normalizada (standard) entre o controlador e o *switch*
- O *switch* pode suportar mais que um canal de controlo em simultâneo, para múltiplos controladores
- A comunicação com o plano de dados é dependente da implementação e interna ao switch

- **Suporta três tipos de mensagens e múltiplos subtipos:**

- **Controller-to-switch**: iniciadas pelo controlador para monitorizar ou atuar sobre o *switch*
- **Asynchronous**: iniciadas pelo *switch* e usadas para notificar o controlador de eventos internos de mudança de estado
- **Symmetric**: iniciadas quer pelo *switch* quer pelo controlador sem qualquer solicitação prévia

OpenFlow – Protocolo



- **As mensagens identificam-se pelo tipo no cabeçalho**

```
/* Header on all OpenFlow packets. */
struct ofp_header {
    uint8_t version;    /* OFP_VERSION. */
    uint8_t type;        /* One of the OFPT_ constants. */
    uint16_t length;     /* Length including this ofp_header. */
    uint32_t xid;        /* Transaction id associated with this packet.
                          Replies use the same id as was in the request
                          to facilitate pairing. */
};
OFP_ASSERT(sizeof(struct ofp_header) == 8);
```

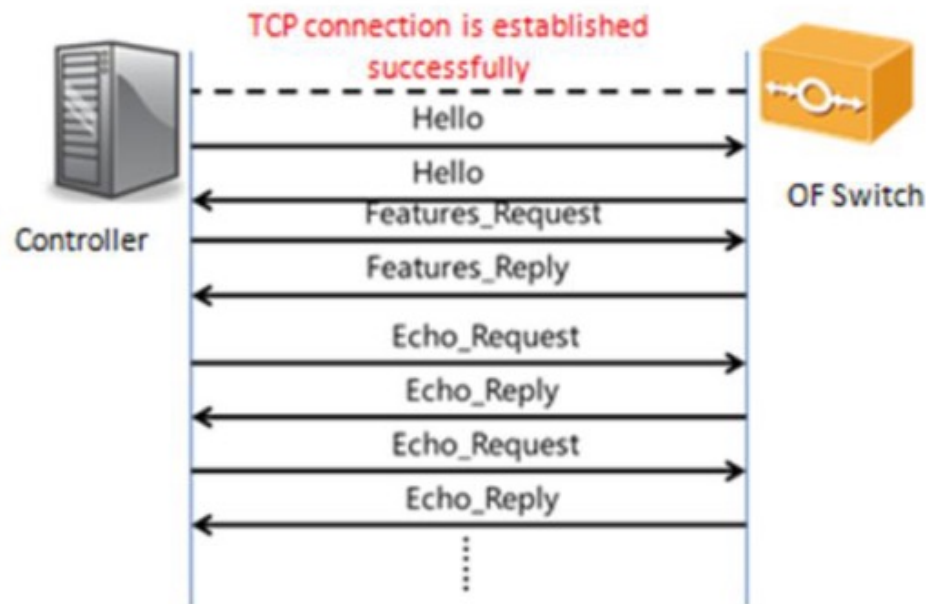
- **Mensagens que iniciam e mantêm a conexão ativa:**

```
enum ofp_type {
    /* Immutable messages. */
    OFPT_HELLO          - 0,  /* Symmetric message */
    OFPT_ERROR           - 1,  /* Symmetric message */
    OFPT_ECHO_REQUEST    - 2,  /* Symmetric message */
    OFPT_ECHO_REPLY      - 3,  /* Symmetric message */
    OFPT_EXPERIMENTER    - 4,  /* Symmetric message */
};
```

OpenFlow – Protocolo



- **OpenFlow Channel handshake:**



- **URLs (porta standard 6653)**

- `tcp:198.168.10.98:6653`
- `tcp:198.168.10.98`
- `tls:test.opennetworking.org:6653`
- `tls:[3ffe:2a00:100:7031::1]:6653`

OpenFlow – Protocolo



- **OpenFlow Channel handshake**

```
/* Switch features. */
struct ofp_switch_features {
    struct ofp_header header;
    uint64_t datapath_id; /* Datapath unique ID. The lower 48-bits are for
                           a MAC address, while the upper 16-bits are
                           implementer-defined. */

    uint32_t n_buffers; /* Max packets buffered at once. */

    uint8_t n_tables; /* Number of tables supported by datapath. */
    uint8_t auxiliary_id; /* Identify auxiliary connections */
    uint8_t pad[2]; /* Align to 64-bits. */

    /* Features. */
    uint32_t capabilities; /* Bitmap of support "ofp_capabilities". */
    uint32_t reserved;
};

/* Capabilities supported by the datapath. */
enum ofp_capabilities {
    OFPC_FLOW_STATS = 1 << 0, /* Flow statistics. */
    OFPC_TABLE_STATS = 1 << 1, /* Table statistics. */
    OFPC_PORT_STATS = 1 << 2, /* Port statistics. */
    OFPC_GROUP_STATS = 1 << 3, /* Group statistics. */
    OFPC_IP_REASM = 1 << 5, /* Can reassemble IP fragments. */
    OFPC_QUEUE_STATS = 1 << 6, /* Queue statistics. */
    OFPC_PORT_BLOCKED = 1 << 8, /* Switch will block looping ports. */
    OFPC_BUNDLES = 1 << 9, /* Switch supports bundles. */
    OFPC_FLOW_MONITORING = 1 << 10, /* Switch supports flow monitoring. */
};
```

OpenFlow – Protocolo



- **OpenFlow message Types (visão parcial apenas):**

```
enum ofp_type {  
    /* Immutable messages. */  
    OFPT_HELLO          - 0,  /* Symmetric message */  
    OFPT_ERROR          - 1,  /* Symmetric message */  
    OFPT_ECHO_REQUEST    - 2,  /* Symmetric message */  
    OFPT_ECHO_REPLY      - 3,  /* Symmetric message */  
    OFPT_EXPERIMENTER    - 4,  /* Symmetric message */  
  
    /* Switch configuration messages. */  
    OFPT_FEATURES_REQUEST - 5,  /* Controller/switch message */  
    OFPT_FEATURES_REPLY   - 6,  /* Controller/switch message */  
    OFPT_GET_CONFIG_REQUEST - 7, /* Controller/switch message */  
    OFPT_GET_CONFIG_REPLY  - 8,  /* Controller/switch message */  
    OFPT_SET_CONFIG        - 9,  /* Controller/switch message */  
  
    /* Asynchronous messages. */  
    OFPT_PACKET_IN        - 10, /* Async message */  
    OFPT_FLOW_REMOVED     - 11, /* Async message */  
    OFPT_PORT_STATUS      - 12, /* Async message */  
  
    /* Controller command messages. */  
    OFPT_PACKET_OUT       - 13, /* Controller/switch message */  
    OFPT_FLOW_MOD          - 14, /* Controller/switch message */  
    OFPT_GROUP_MOD        - 15, /* Controller/switch message */  
    OFPT_PORT_MOD         - 16, /* Controller/switch message */  
    OFPT_TABLE_MOD        - 17, /* Controller/switch message */  
}
```